Solomonoff Function Induction

Kaarel Hänni

September 2025

1 Introduction

The following setup is standard in machine learning.

- One has a *training data set* $\mathcal{D} \subseteq X \times Y$ — that is, a set $\mathcal{D} = \{(x_i, y_i) : i \in I\}$ of things $x_i \in X$ ("inputs") with associated things $y_i \in Y$ ("outputs" or "labels").[1]
  - an example: We could have each x_i be either a picture of a cat or a picture of a dog, with each label y_i being either "cat" or "dog" correspondingly.
  - another example: We could have

    $$\mathcal{D} = \{(n, \pi(n)) : 1 \leq n \leq 11\}$$

    $$= \{(1,0), (2,1), (3,2), (4,2), (5,3), (6,3), (7,4), (8,4), (9,4), (10,4), (11,5)\},$$

    where $\pi(n)$ is the prime counting function.
- One wishes to use knowledge of this training data set $\mathcal{D}$ to assign labels to inputs $x \in X$, including to those that do not show up in the training data set $\mathcal{D}$. In other words, one wishes to carry out a kind of *induction* — one wishes to guess "unseen" behavior from "seen" behavior.
  - an example: Suppose you are given a data set of the one finds when searching the internet and a data set of the first 100 salamander pictures one finds when searching the internet, correspondingly with labels "A" or "B". Like, you're given a stack of 200 pieces of paper with the pictures and respective labels. Suppose that a coin is then flipped and you are then shown either the 101st lizard picture or the

[1] Technically, we will assume (perhaps non-standardly for machine learning, but arguably with minimal loss of generality) that all x_i and y_i are ultimately given as finite bitstrings; when we speak of them as being other kinds of things — e.g., digital texts or sound sequences or video sequences, or elements of some finite set of labels — we're implicitly assuming that some simple way to encode these other kinds of things as bitstrings has been fixed. (Alternatively, one could go with the convention that x_i and y_i are ultimately natural numbers — this would be equivalent for our purposes.)

101st salamander picture, and asked to assign it "A" or "B" (trying to "continue the pattern"). Probably, most psychologically normal humans are going to "get this right", i.e. assign "A" if shown a lizard and assign "B" if shown a salamander.

- One can also wish to guess the

  make a machine M that assigns labels to inputs x and that does so *well* — in particular, M should assign labels well even to inputs x which do not show up in $\mathcal{D}$. To be precise, we will say that M should be a Turing machine and

There is a very natural ideal thing of this kind: you take all

2 Fundamental properties

For solomonoff induction type things, it's generally nice to have the following kind of theorem. Thm 1 (template). For a given class C of predictors, each predictor P in C loses to "the solomonoff inductor for C". That is: on any data distribution, that solomonoff inductor does worse than P in log loss by at most a constant (depending on P but not the distribution).

For solomonoff function induction, I think there are some subtleties with getting this sort of result. The point of this note is to discuss that. If anything is unclear, I'm happy to try to explain better. (There are a few things here that I haven't thought through completely carefully, so there's also some chance I'm wrong about a few things.)

So, as before, we're in the setting where you have input-output pairs from a function f: X-¿Y revealed, and your job is to predict outputs on new inputs.

I think a Thm 1 type statement is easy to prove if we take C to be the set of all predictors P given by turing machines that get to see an input x and have to compute a probability distribution on Y — i.e., those P that can be specified as follows: * Specify a turing machine T; on a given input x in X and output y in Y, it is supposed to output a positive rational number (well, we assume some reasonable way to represent positive rationals as bitstrings has been specified, and this is how we interpret its output) and these rationals must sum up to 1. Actually, we can prove Claim 1 for *computably samplable* predictors — i.e, predictors specified implicitly in the form of a computable sampler as follows:

1. Specify a probabilistic turing machine T that gets an input x as its input and outputs some output y in Y.

2. It's even fine to allow T which don't always halt here; we think of those as implicitly giving a measure summing to less than 1 (`https://en.wikipedia.org/wiki/Sub-probability_measure`).

If a probability distribution is computable (in the first sense above), then one can computably sample from it. Thus, if we show that any computably samplable predictor loses to solomonoff function induction, then we've also shown

that any computable distribution loses to it. And any computably samplable predictor indeed loses to (the universal semimeasure version) of solomonoff function induction basically because of Claim 1 from earlier (a small technicality here: one should use different bits of randomness from the universal semimeasure's infinite read tape for sampling on different inputs; one can do this using dovetailing). So we get a version of Thm 1.

This version of Thm 1 feels a bit unsatisfying to me, because arguably the most natural relevant class of computable predictors to compare solomonoff function induction against is not the one considered above — it is arguably more natural to let a predictor see the observed input-output data set so far when making each successive prediction. Unfortunately, if the order in which input-output pairs are revealed can have an arbitrary distribution (and especially if there can be a correlation between this order and the function), then I think Thm 1 style statements are just false. Vaguely this is because: there are computable predictions which make use of information about which x's had their outputs revealed previously to make predictions, which isn't something solomonoff function induction can natively do, and one can construct data sequences where access to previous input information lets you do really well but on which solomonoff function induction struggles to do well. Concretely, let $x_1, x_2, \ldots$ be a deterministic sequence of increasingly long random strings, and make a function which has input-output pairs $(x_1, 0), (x_2, x_1), (x_3, x_2), (x_4, x_3), \ldots$, and make the data set reveal them in precisely that order. Then we have the following very simple computable predictor that gets 0 loss: predict 0 initially, and then predict whichever string appears in the data set as an input but not as an output. However, I'm pretty sure Solomonoff induction gets unbounded loss on this sequence. I don't actually have a proof of this latter claim, but I have another slightly more complicated construction for which I can prove that there is a simple computable predictor that gets 0 loss but solomonoff must get unbounded loss. (I can provide this other example if there is interest.)

I think we've learned from the above that there can't be a version of Thm 1 which picks a constant first and then quantifies universally over [distributions on the sequence of revealed inputs], and in fact that there are even particular reveal input sequence distributions Δ such that there can't be a version of Thm 1 even with Δ fixed as the reveal sequence distribution. One could hope that there would at least be a version for simple reveal sequence distributions, but I think this will also fail if the data-generating process is allowed to have the reveal sequence choice provide information about the function — actually, one can just use the random string example above, except now putting the randomization inside the reveal sequence distribution, and letting the function depend on the chosen reveal sequence as stated above. However, we can get a version of Thm 1 in case the data-generating process is such that the reveal sequence distribution is easy to sample from conditional on any particular choice of function f: ¿ Thm 1. Let C be the set of computable samplers which get to see the revealed input-output data set so far when sampling the output on the next input (we take these samplers to implicitly specify predictors, as before). Suppose the data-generating process samples a function $f: X \to Y$ and then reveals a sequence

of input-output pairs, with the sequence of inputs sampled from a distribution Δ. Then each predictor P in C can only beat solomonoff function induction in expected log loss by at most the complexity C_Δ of sampling from the reveal sequence distribution Δ plus a constant C_P (depending on the predictor but not the data-generating process).

Pf sketch: Inside solomonoff function induction, one has a probabilistic turing machine that "internally simulates" sampling a first input x_1 using Δ, then sampling an output y_1 on x_1 using P, then sampling a second input x_2 using Δ (which can take into account (x_1, y_1)), then sampling an output y_2 on x_2 using P (which can take into account (x_1, y_1) and x_2), and so on, until it happens to sample the current input x, samples an output y for it, at which point it outputs that. One can analyze the difference between this "internally simulated sampling from Δ and P" and what P is actually doing as it predicts outputs of f on inputs from Δ, and conclude that the expected log loss of the former is at most the expected log loss of the latter. (Sketch: use linearity of expectation to reduce it to proving a bound for the expected log loss at each input x, and then use jensen's inequality to say that averaging your predictions before calculating loss beats averaging your losses (noting that the internal simulator is indeed averaging predictions over all possible paths to the input x).)

2.1 Input encoding doesn't matter much

input encoding doesn't matter much becasue

2.2 Equivalence between Kolmogorov version and universal semimeasure version

need coding theorem

3 A question about generalizing out of distribution

A question that often in machine learning is whether a certain model would generalize "well", especially "out of distribution" — that is, let us say, to inputs meaningfully unlike those on which the model was trained.[2] Here are two reasonable meanings of "generalizing well":

1. Sometimes, good generalization means generalization in some specific way — like, we could hope that a model trained to assign truth values correctly to (provable or disprovable) statements in number theory would generalize to assigning truth values correctly to (provable or disprovable) statements in geometry.

[2]It is probably reasonable to think of there being a spectrum from generalizing well on inputs which are very much like the inputs a model was trained on to inputs which are very different from inputs a model was trained on.

2. Other times, the point is less to generalize in some unique way, and more to produce more outputs with some property (that often cannot be explicitly defined).[3] For example, you could pretrain or fine-tune a language model on a bunch of mathematical text, and then hope that when prompted with "Theorem. [insert a mathematical statement here]. Proof.", it provides a correct proof of the theorem. This is distinct from the criterion of correct generalization being the provision of a specific proof — we're quite happy with any correct proof being provided in this case.

Really, the first notion is a special case of the second, just with the property that the input-output pairs are those from a particular function. So it will be reasonable for our purposes to proceed from here with the second notion of good generalization as the general case.

result:

- the probability of P-abiding generalization is at least $2^{-C(P)}$
- if no input with property Q is seen during training, then probability of P-abiding generalization is at most $2^{-C(Q)}$ times 2 to the negative complexity of the smallest string not satisfying P on the given input times a const

that is, regardless of how much train data has been seen, good ood generalization to strings with some property has at least a const probability of happening but also at least some const probability of not happening, at least for a loose meaning of 'const'

[3] Or more precisely, to obtain a model which can produce outputs on novel inputs such that the produced input-output pairs have some property.